Week 1: Python Fundamentals
Assignment Report

Name: Yash Kulkarni

Marks: 100

Submission: PDF Report + Google Drive Link

Drive Link: https://drive.google.com/file/d/1Y5bK0zY95m5xDiltfgin1bW0m1TK0uAo/view?usp=drive_link

Task	Title	Marks
1	Python Installation & Setup	15
2	Variables & Data Types	20
3	Input, Output & Type Casting	20
4	Operators in Python (Calculator)	20
5	Mini Program - Temperature Converter	25
	Total	**100**

Introduction

This report documents the Week 1 Python Fundamentals assignment. It covers installing Python, setting up VS Code, variables and data types, input/output, type casting, operators, and a mini program. For every task the report gives a short explanation, the full source code and the program output. All .py files are uploaded to Google Drive (link on the cover page).

Task 1: Python Installation & Setup (15 Marks)

Explanation

Python is a high-level, easy-to-read programming language used in web development, data science, automation and AI. VS Code is a lightweight, free code editor from Microsoft that supports Python through the official Python extension.

Installation steps

1. Install Python: download the latest installer from python.org/downloads, run it and tick **'Add python.exe to PATH'**, then click *Install Now*.
2. Verify: open Command Prompt / PowerShell and run `python --version` and `pip --version`.
3. Install VS Code: download from code.visualstudio.com and install with the default options.
4. Add the Python extension: open Extensions (Ctrl+Shift+X), search 'Python' (by Microsoft) and click Install.
5. Select interpreter: press Ctrl+Shift+P, choose *Python: Select Interpreter* and pick the installed Python.
6. Run first program: create `hello.py`, type `print("Hello, World!")` and press the Run button or run it from the terminal.

Screenshots of setup

Figure 1: Python 3.11.1 installer with 'Add python.exe to PATH' selected

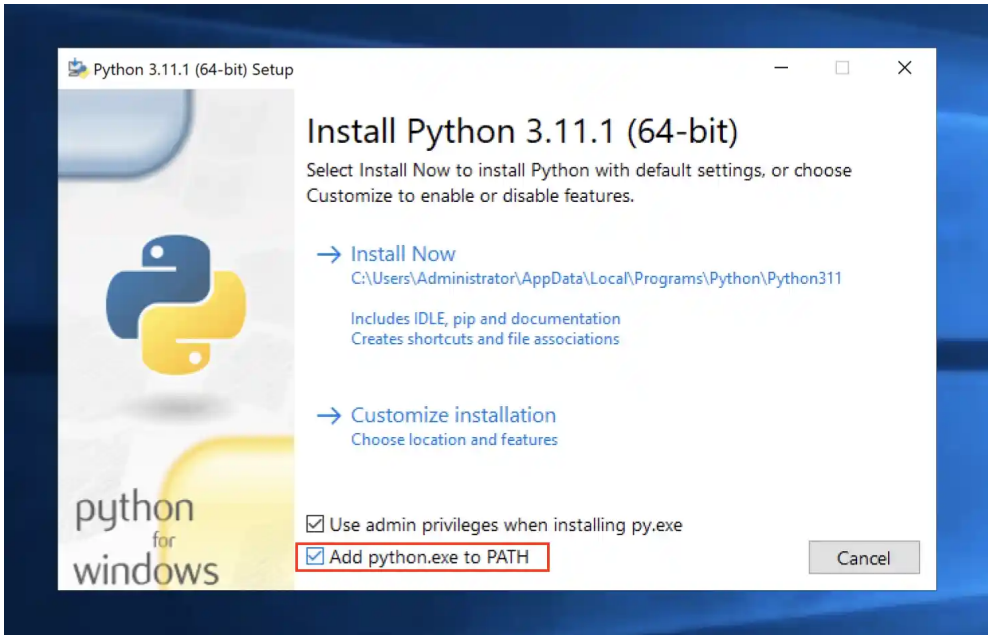


Figure 2: VS Code Extensions view showing the Python extensions

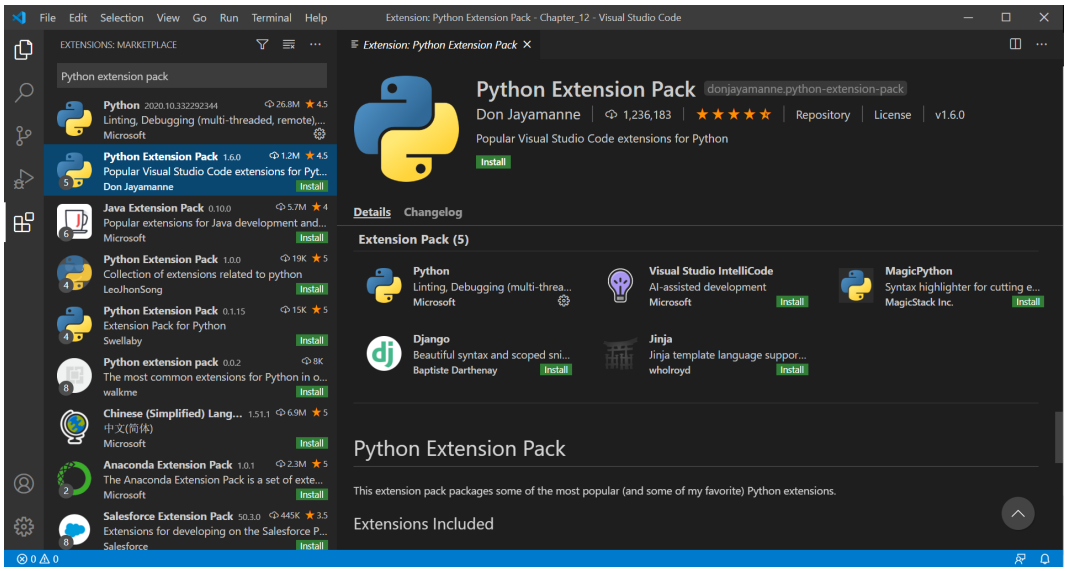
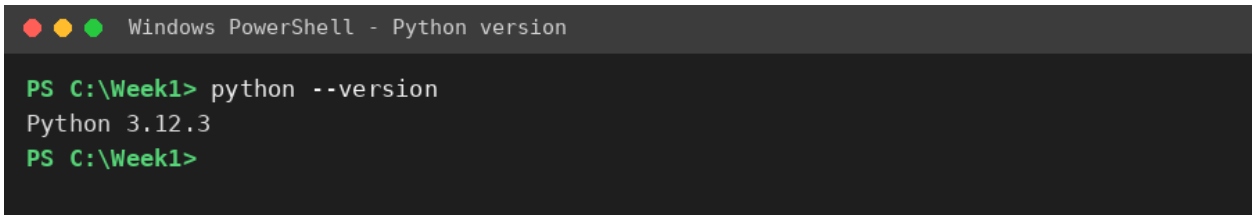
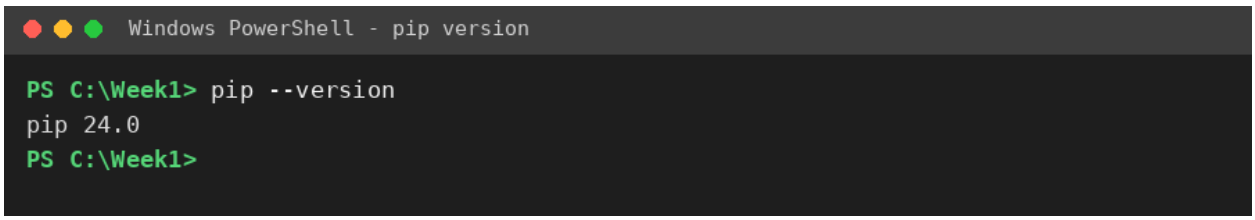


Figure 3: Python version check



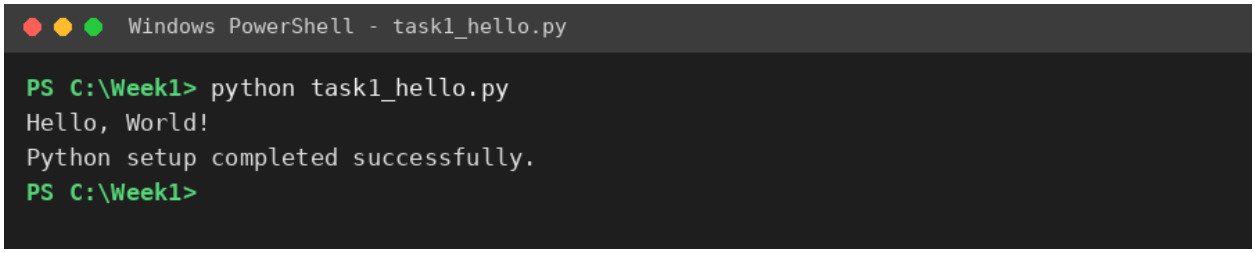
pip version check



First program - Hello, World!

```
# Task 1: First Python program - Hello, World!
print("Hello, World!")
print("Python setup completed successfully.")
```

Output



Task 2: Variables & Data Types (20 Marks)

Explanation

A variable is a named memory location that stores a value. Python is dynamically typed, so the data type is decided automatically from the value. This program declares variables of ten different types (str, int, float, bool, list, tuple, dict, set, complex and NoneType) using meaningful names such as `student_name` and `cgpa`, then prints each value together with its type using the built-in `type()` function.

Source Code (task2_variables.py)

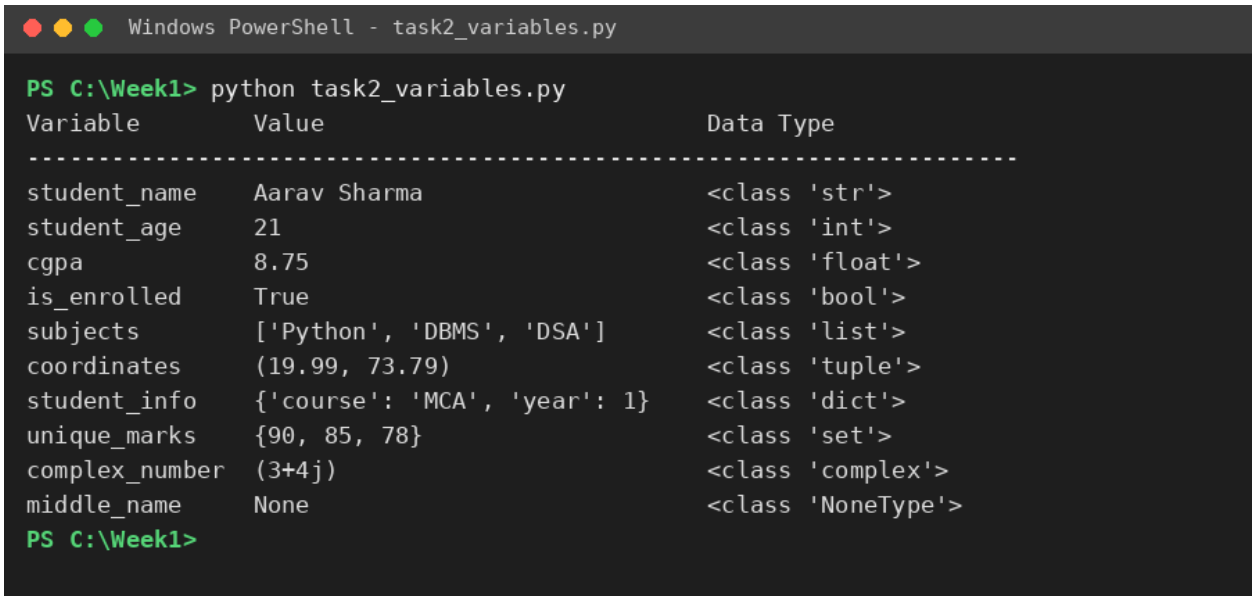
```
# Task 2: Variables and Data Types

student_name = "Aarav Sharma"        # str
student_age = 21                     # int
cgpa = 8.75                          # float
is_enrolled = True                   # bool
subjects = ["Python", "DBMS", "DSA"] # list
coordinates = (19.99, 73.79)         # tuple
student_info = {"course": "MCA", "year": 1}  # dict
unique_marks = {85, 90, 78}          # set
complex_number = 3 + 4j              # complex
middle_name = None                   # NoneType

variables = {
    "student_name": student_name,
    "student_age": student_age,
    "cgpa": cgpa,
    "is_enrolled": is_enrolled,
    "subjects": subjects,
    "coordinates": coordinates,
    "student_info": student_info,
    "unique_marks": unique_marks,
    "complex_number": complex_number,
    "middle_name": middle_name,
}

print(f"{'Variable':<16}{'Value':<32}Data Type")
print("-" * 70)
for name, value in variables.items():
    print(f"{name:<16}{str(value):<32}{type(value)}")
```

Output


Windows PowerShell - task2_variables.py
PS C:\Week1> python task2_variables.py
Variable Value Data Type
student_name Aarav Sharma <class 'str'>
student_age 21 <class 'int'>
cgpa 8.75 <class 'float'>
is_enrolled True <class 'bool'>
subjects ['Python', 'DBMS', 'DSA'] <class 'list'>
coordinates (19.99, 73.79) <class 'tuple'>
student_info {'course': 'MCA', 'year': 1} <class 'dict'>
unique_marks {90, 85, 78} <class 'set'>
complex_number (3+4j) <class 'complex'>
middle_name None <class 'NoneType'>
PS C:\Week1>

Task 3: Input, Output & Type Casting (20 Marks)

Explanation

The `input()` function reads data from the keyboard and always returns a string. Since age must be used as a number, it is converted using `int()` (type casting). The program prints the type before and after casting to show the change, and then displays a formatted introduction using f-strings.

Source Code (task3_intro.py)

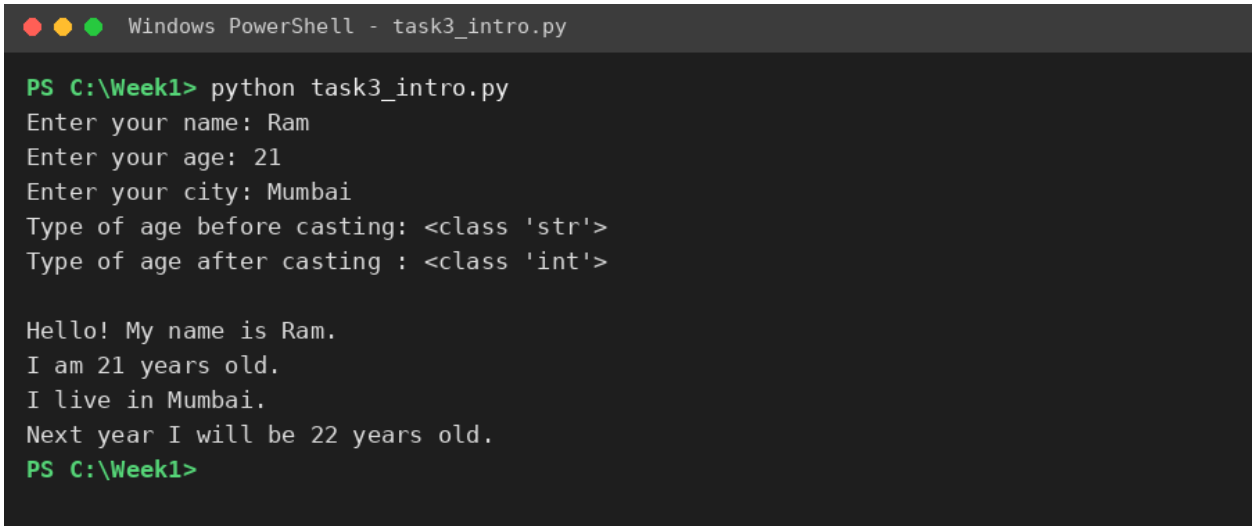
```
# Task 3: Input, Output and Type Casting

name = input("Enter your name: ")
age_text = input("Enter your age: ")
city = input("Enter your city: ")

print("Type of age before casting:", type(age_text))
age = int(age_text)                      # type casting str -> int
print("Type of age after casting :", type(age))

print()
print(f"Hello! My name is {name}.")
print(f"I am {age} years old.")
print(f"I live in {city}.")
print(f"Next year I will be {age + 1} years old.")
```

Output



Task 4: Operators in Python (Calculator) (20 Marks)

Explanation

This calculator takes two numbers from the user (converted to float) and applies the arithmetic operators `+ - * / % **` for addition, subtraction, multiplication, division, modulus and exponent. A check is added so that division and modulus by zero do not crash the program.

Source Code (task4_calculator.py)

```
# Task 4: Calculator using operators

num1 = float(input("Enter first number : "))
num2 = float(input("Enter second number: "))

print("\n----- Results -----")
print(f"Addition       : {num1} + {num2} = {num1 + num2}")
print(f"Subtraction    : {num1} - {num2} = {num1 - num2}")
print(f"Multiplication : {num1} * {num2} = {num1 * num2}")

if num2 != 0:
    print(f"Division       : {num1} / {num2} = {num1 / num2}")
    print(f"Modulus        : {num1} % {num2} = {num1 % num2}")
else:
    print("Division       : Not possible (divide by zero)")
    print("Modulus        : Not possible (divide by zero)")

print(f"Exponent       : {num1} ** {num2} = {num1 ** num2}")
```

Output

```
Windows PowerShell - task4_calculator.py

PS C:\Week1> python task4_calculator.py
Enter first number : 20
Enter second number: 3

----- Results -----
Addition       : 20.0 + 3.0 = 23.0
Subtraction    : 20.0 - 3.0 = 17.0
Multiplication : 20.0 * 3.0 = 60.0
Division       : 20.0 / 3.0 = 6.666666666666667
Modulus        : 20.0 % 3.0 = 2.0
Exponent       : 20.0 ** 3.0 = 8000.0
PS C:\Week1>
```

Task 5: Mini Program - Temperature Converter (25 Marks)

Explanation

The Temperature Converter shows a menu and converts between Celsius, Fahrenheit and Kelvin. It uses variables, `input()`, type casting (`int()` for the menu choice and `float()` for the temperature), arithmetic operators and if-elif-else decision making. Formulas: F = (C x 9/5) + 32, C = (F - 32) x 5/9, K = C + 273.15.

Source Code (task5_temperature_converter.py)

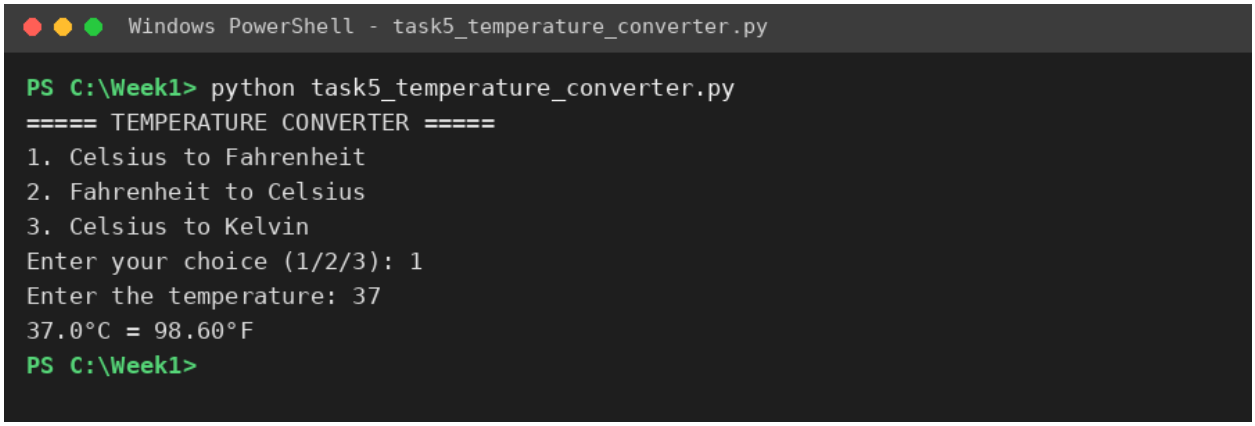
```
# Task 5: Mini Program - Temperature Converter
# Uses variables, input, operators and type casting.

print("===== TEMPERATURE CONVERTER =====")
print("1. Celsius to Fahrenheit")
print("2. Fahrenheit to Celsius")
print("3. Celsius to Kelvin")

choice = int(input("Enter your choice (1/2/3): "))
temp = float(input("Enter the temperature: "))

if choice == 1:
    result = (temp * 9 / 5) + 32
    print(f"{temp}°C = {result:.2f}°F")
elif choice == 2:
    result = (temp - 32) * 5 / 9
    print(f"{temp}°F = {result:.2f}°C")
elif choice == 3:
    result = temp + 273.15
    print(f"{temp}°C = {result:.2f} K")
else:
    print("Invalid choice. Please enter 1, 2 or 3.")
```

Output



Conclusion

In Week 1, Python was installed and configured, and five programs were written covering variables, data types, input/output, type casting and operators. These programs build the base needed for conditions, loops and functions in the coming weeks.